

Aufgabe 1: Schmucknachrichten

Teilname-ID: 74314

Bearbeiter/-innen dieser Aufgabe:

Tao Zheng

26. Dezember 2024

Inhaltsverzeichnis

1 Annahmen	1
2 Lösungsidee	1
2.1 Zusätzliche Funktion für Aufgabe a: Huffman Codierung	2
3 Umsetzung	2
3.1 Umsetzung der Huffman-Codierung	3
3.2 Main Funktion	4
4 Analyse des Funktions	4
4.1 Analyse der Problemstellung	5
5 Beispiele	6
6 Quellcode	6
6.1 Huffman-Codierung	8

1 Annahmen

Die Perlendurchmesser (aus den Dateien) sind immer aufsteigend sortiert.

Alle Perlen haben einen Durchmesser von mind. 1 mm.

Es gibt immer mind. 2 verschiedene Perlen.

Alle Dateien haben das richtige Format, das im README.txt angegeben ist.

2 Lösungsidee

Für die Idee wird die Anzahl aller Buchstaben/Symbole gezählt. Das liegt daran, dass die Endgröße unabhängig von der Position sowie Kombination der Eingaben ist, sondern nur von der Häufigkeit der Buchstaben des eingegebenen Textes abhängt. Nach dem Zählen der Häufigkeiten eines Buchstabens nimmt man die Häufigkeiten aller Symbole her und generiert dadurch die Kodierung, die man später den Symbolen zuweist.

Die Häufigkeiten werden dabei in eine rekursive Funktion eingesetzt, die proportional zur Anzahl der verschiedenen Perlen die einzelnen Häufigkeiten in verschiedene Gruppen aufteilt. Jede Gruppe sowie Untergruppe stellt eine Perlenart dar, die für die Codierung verwendet wird. Diese aufgeteilten Gruppen werden wieder in weitere Untergruppen aufgeteilt, bis die Untergruppen nur noch aus einem einzigen Häufigkeitselement bestehen. Dadurch ergibt sich eine Struktur, die mehrere Häufigkeiten in mehrere Untergruppen verteilt hat, und die Reihenfolge sowie die Verschachtelung der Liste ergeben zum Schluss eine Kodierungsliste, die noch keinen Symbolen zugewiesen ist.

Bei der Aufteilung sollen die Häufigkeiten abhängig vom Perlendurchmesser verteilt werden, wobei Perlen mit geringem Durchmesser mehr Häufigkeiten aufnehmen sollen, während Perlen mit größerem Durchmesser weniger Häufigkeiten aufnehmen sollen.

Sobald man die Kodierungsliste erhalten hat, berechnet man für alle Kodierungen die Gesamtlänge der Perlen, die für eine Kodierung benötigt wird, und sortiert alle Gesamtlängen aufsteigend. Zusätzlich werden Symbole in eine absteigende Reihenfolge anhand ihrer Häufigkeit sortiert und anschließend einer

Kodierung zugewiesen. Da die Kodierung aufsteigend sortiert ist und die Symbole mit ihren Häufigkeiten absteigend sortiert sind, bekommen Symbole mit hoher Häufigkeit kürzere Kodierungen zugewiesen und Symbole mit geringer Häufigkeit längere Kodierungen zugewiesen.

Zum Schluss wird für die Berechnung der Gesamtlänge der Kette für alle Perlenarten sowie Symbole das Produkt aus Häufigkeit der Perlenart in den Symbolen, Häufigkeit des Symbols sowie die Perlendurchmesser der Perlenart berechnet, alle Produkte werden aufsummiert und zur Längeneinheit cm mit 1cm/10 Perlen umgewandelt.

2.1 Zusätzliche Funktion für Aufgabe a: Huffman Codierung

Für die Berechnung einer Präfix-Codierung für alle Schmuckaufgaben, die den gleichen Perlendurchmesser besitzen, möchte ich die Huffman Codierung verwenden. Die Huffman Codierung garantiert, solange es keine unterschiedlichen Perlendurchmesser existiert, den optimalen Präfixcode. In der Huffman Codierung bildet man eine Baumstruktur durch das Anknüpfen von Blattknoten bzw. in diesem Fall: die Symbole. Dabei nimmt man einen Stapelspeicher, der immer die Gruppen/Symbole nach der Häufigkeit sortiert. Abhängig von der Anzahl der Perlenarten werden mehrere Symbole/Gruppen (Child-Knoten) mit den geringsten Häufigkeiten aus dem Stapelspeicher entnommen und zu einer Gruppe (Parent-Knoten) gebildet (die Gruppen werden wie ein Symbol behandelt), deren Häufigkeit die Summe aller Häufigkeiten der Symbole in der Gruppe ist. Diese Gruppe wird in den Stapelspeicher eingefügt und der Stapelspeicher wird wieder nach den Häufigkeiten aufsteigend sortiert. Dieser Schritt wird so lange wiederholt, bis es zum Schluss nur noch eine Gruppe gibt, welche unsere Wurzel im Baumdiagramm ist.

Mithilfe der Formel:

$$(\text{Länge von Anzahl der Zeichen} - 2) \% (\text{Anzahl der verschiedenen Perlen} - 1) + 2 \quad (1)$$

wird die maximale Gruppengröße für die allererste Gruppe bestimmt. Durch diese Beeinflussung wird garantiert, dass die letzte Gruppe bzw. die Wurzel die maximale Anzahl von Besetzungen haben wird und man am Ende eine optimale Kodierung erhält.

Anhand der Baumstruktur sollen zum Schluss die Kodierungen abgelesen werden, indem man vom Wurzelknoten durch alle verknüpften Child-Knoten geht und dabei den benötigten Indexen bis zum Blattknoten merkt und beim Blattknoten zusammen mit seinem Symbol ausdrückt.

Für die Berechnung der Kettenlänge werden alle Child-Knoten durchgegangen und dabei die Entfernung zwischen Wurzelknoten und Blattknoten berechnet. Anschließend wird diese Entfernung mit der Häufigkeit des Symbols multipliziert und für alle Blattknoten aufsummiert.

3 Umsetzung

Für die Umsetzung habe ich es in drei verschiedene Funktionen aufgeteilt:

- GetPrefix (berechnet die Kodierungen von der zugewiesenen Gruppe rekursiv und erzeugt anschließend die Kodierungen mit ihrer Kodierungslänge. Argumente: Kodierungen als Dictionary (codes), die Häufigkeiten als Liste (amounts), bisherige Kodierungen als String (prefix), die Durchmesser aller Perlen als Liste (dPerle) und optional die bisherige Länge als Integer.)
- TotalSize (berechnet die Gesamtlänge der Kette. Argumente: Die Präfixcodes als Dictionary, Häufigkeiten aller Buchstaben als Dictionary (charmap) sowie die Perlendurchmesser als Liste (dPerle). Diese Funktion gibt die Gesamtzahl der benötigten Perlen als Integer und die Länge der Kette als String zurück.)
- Hauptfunktion Main (Eintrittspunkt. Argument: Dateiname als String.)

Im Main wird zunächst einmal die Datei abgelesen. Mithilfe der Kodierung „utf-8“ können alle Symbole abgelesen werden, u.a. chinesische und japanische Symbole. Dabei werden die Perlendurchmesser in der Liste „dPerle“ gespeichert und alle Einträge zu Integern umgewandelt. Die Botschaft wird in der Variable sentence als String gespeichert, um später zu zählen.

Anschließend werden alle Buchstaben auf ihre Häufigkeit in der Botschaft gezählt und gespeichert. Für die Zählungen verwende ich ein Dictionary (charmap), das als Key das Symbol als String speichert und

die Häufigkeit als Integer, und eine Hashmap (charset), die zur Überprüfung verwendet wird, ob ein Eintrag für den Buchstaben schon erstellt wurde. Ich habe mich für die Hashmap entschieden, da durch die Überprüfung in der Hashmap eine durchschnittliche Laufzeit von $O(1)$ garantiert wird und es deshalb in der Theorie besser für die Laufzeit bei großer Datenmenge ist. Für jeden Buchstaben wird daher die Anzahl der Häufigkeit für den bestimmten Buchstaben um eins erhöht und falls ein Buchstabe zum ersten Mal auftaucht, wird es in charset sowie charmap (mit dem Wert 1) aufgenommen.

Sobald alle Symbole gezählt wurden, werden alle Dictionary Einträge von charmap zur Liste aus Tuples umgewandelt, wobei an der ersten Position das Symbol ist und an der zweiten Position die Häufigkeit des Symbols. Diese Liste wird verwendet, um die einzelnen Tuples nach ihrer Häufigkeit absteigend zu sortieren und anschließend unter Einhaltung der Reihenfolge die Häufigkeiten und Symbole voneinander zu trennen und in zwei Listen (amounts und symbols) zu speichern.

Im getPrefix wird anschließend die Kodierung berechnet. Dabei stellt das Auslösen der Funktion eine Perle dar, die abhängig von der Anzahl der Perlen einer Perle max. Eine Häufigkeit zuweist. Wenn es mehr als einen Häufigkeitswert in der Gruppe gibt, werden für die nächsten Perlen so viele Listen wie die Anzahl der verschiedenen Perlen erstellt. Diese Listen stellen die neuen Gruppen dar und werden in einer Liste (groups) aufbewahrt, wodurch eine 2-dimensionale Liste entsteht. Um die Häufigkeiten zu verteilen, wird eine Liste aus Integern erstellt, die so viele Einträge besitzt, wie es verschiedene Perlen gibt. Diese Liste aus Integern stellt die Gesamtzahl der Häufigkeiten, multipliziert mit dem Perlendurchmesser, dar und ich nenne es deshalb Gewicht, da die Verteilung von den zwei genannten Faktoren abhängt.

Anschließend werden alle Häufigkeiten in die Gruppen verteilt: Dabei werden die Häufigkeiten, die absteigend sortiert sind, in die erste Gruppe mit dem niedrigsten Gewicht aufgenommen. Die Häufigkeit wird anschließend multipliziert mit dem Perlendurchmesser in dem Gewicht addiert. Sobald alle Häufigkeiten zugewiesen wurden, wird für alle erstellten Gruppen diese Funktion erneut ausgelöst. Dabei werden für die darauffolgenden Funktionsaufrufe die bisherigen Kodierungen mit der Position der Gruppe aufaddiert und das Gesamtgewicht im Bereich um den Perlendurchmesser erhöht. Somit werden die Kodierungen rekursiv berechnet. Sobald bei einer Verteilung nur noch eine Häufigkeit in der Gruppe vorhanden ist, wird diese Kodierung mit dem Gesamtgewicht in codes gespeichert.

Sobald alle Kodierungen berechnet wurden, werden die Kodierungen nach ihrer Gesamtgröße aufsteigend sortiert (codesorted). In einer separaten Variable NewCode werden dann die neuen Kodierungen gespeichert, indem man die Symbole, die zuvor absteigend sortiert waren, einer Kodierung, die aufsteigend sortiert ist, zuweist.

In totalSize wird anschließend die Länge der Kette berechnet. In zwei for-Schleifen werden für alle Kodierungen, Perlenarten und Häufigkeiten in Integer total aufsummiert. Dies wird folgendermaßen erreicht: (Perlenart steht für eine Zahl zwischen 0 und der Anzahl der verschiedenen Perlen.)

$$\text{Symbol.Kodierung.zählealle(Perlenart)} \cdot \text{Symbol.Häufigkeit} \cdot \text{Perlendurchmesser[Perlenart]} \quad (2)$$

Die Gesamtgröße der Kette wird anschließend zurückgegeben und mithilfe einer for-Schleife werden für alle Präfixcodes das Symbol und der Wert ausgegeben.

3.1 Umsetzung der Huffman-Codierung

Die Umsetzung ist auf eine Huffman-Codierung basiertes Programm angelehnt. Dabei wird ein Baum erstellt, dessen Knoten, proportional zu der Perlenanzahl, Nachfolger (childrens, als Liste) aufnehmen kann. Für das Erstellen der Klasse Knoten werden Symbol (String oder None), Häufigkeit (Integer) und als optionale Möglichkeit Child-Knoten (Liste) aufgenommen. Diese gegebenen Daten werden in der Klasse gespeichert. Hinzu wird noch eine Vergleichsfunktion (`__lt__`) zu der Klasse hinzugefügt, um Sortierungen zu behandeln. Wenn zwei Knotenklassen miteinander verglichen werden, wird es anhand der Häufigkeit verglichen.

Für die Berechnung der Gesamtlänge wird die rekursive Funktion „totalSize“ erstellt. Diese berechnet die Summe, indem man vom Wurzelknoten aus bei allen Child-Knoten diese Funktion aufruft und dabei die Anzahl der Kanten vom Wurzelknoten zum Child-Knoten weitergibt. Der Rückgabewert der Funktion

wird in einer Variablen Summe gespeichert, welche nach der letzten Child-Knoten zurückgegeben wird. Sobald es bei einem Blattknoten ankommt, wird die Anzahl der Kanten mit der Häufigkeit des Symbols multipliziert und zurückgegeben.

Im `printOut` bzw. `strOut` werden die Huffman-Codes ausgegeben bzw. zurückgegeben. Dabei geht man vom Wurzelknoten aus durch alle Child-Knoten und löst diese Funktion im Child-Knoten aus. Beim Auslösen der Funktion wird zusätzlich noch die Position der Knoten im Parent-Knoten weitergegeben, die sich später zu einer vollständigen Kodierung bildet. Sobald man an einen Blattknoten ankommt, wird ein Text erstellt, der an den Parent-Knoten zurückgegeben wird. Beim Parent-Knoten werden alle Texte zusammengefügt und zurückgegeben.

3.2 Main Funktion

Im Main werden zunächst einmal die Daten aus der Datei abgelesen und die Anzahl der Perlen (`nPerle`), der Durchmesser der Perlen (`dPerle`) und der Satz gespeichert. Anschließend werden wieder alle Buchstaben mithilfe einer Dictionary (`charmap`) und einer Hashmap (`charset`) gezählt.

Mithilfe der genannten Formel wird nun die Gruppengröße als Integer für die allererste Gruppe berechnet (Joker).

Alle Symbole werden mit ihren Häufigkeiten in einen Knoten umgewandelt, welcher in der Liste Knoten gespeichert wird, und die Knoten werden nach ihren Häufigkeiten absteigend sortiert (da `pop` immer das letzte Element ausgibt).

Anschließend wird, solange sich mehr als ein Knoten im sortierten Stapelspeicher befindet, abhängig von der Gruppengröße die Knoten mit der kleinsten Häufigkeit aus dem sortierten Stapelspeicher entnommen und ein neuer Knoten erstellt, dessen Child-Knoten (`childrens`) die entnommenen Knoten sind und dessen Häufigkeit die Summe aller Häufigkeiten der entnommenen Knoten entspricht. Dieser Knoten wird in den Stapelspeicher hinzugefügt und der Stapelspeicher wird anschließend wieder nach seinen Häufigkeiten absteigend sortiert.

Sobald sich nur noch ein Knoten im Stapelspeicher befindet, wird dieser Knoten als Wurzelknoten eines Baumdiagramms gesehen und von dort aus die Gesamtgröße sowie die Kodierungen mithilfe der Funktionen `totalSize` und `strOut` ausgegeben.

4 Analyse des Funktions

Bei der Funktion `getPrefix` handelt es sich um eine rekursive Funktion. Dabei werden die Variablen `groups` und `sums` abhängig von der Anzahl der verschiedenen Perlen (`nP`) erstellt, weshalb eine Laufzeitkomplexität und Speicherkomplexität von $\mathcal{O}(nP)$ für `sums` und `groups` gilt. In der `for`-Schleife werden alle Häufigkeitswerte durchgegangen (`nA`). Dabei wird in der `for`-Schleife der Index aus dem Minimum der `sums` berechnet. Da `sums` von `nP` abhängig ist, gilt für die `for`-Schleife eine Laufzeitkomplexität $\mathcal{O}(nA \cdot (2 \cdot nP))$ was sich zu $\mathcal{O}(nA \cdot nP)$ vereinfachen lässt. Zum Schluss wird in einer `for`-Schleife für alle `nP` diese Funktion erneut ausgelöst. Dabei kann mithilfe der Formel: $nA + (nA - 1) // nP$ ein Näherungswert für den Durchschnitt der Funktionsaufrufe erstellt werden. Der Wert $(nA - 1) // nP$ kommt davon, dass für alle Perlenarten immer mind. ein Wert zugewiesen wird, solange für die Ausführung $nA \geq nP$ gilt. Aus diesem Grund lässt sich daraus $\mathcal{O}(nP \cdot (nA + (nA - 1) // nP))$ bilden, welches sich zu $\mathcal{O}(nP \cdot nA)$ vereinfachen lässt. Unter Beachtung der großen Faktoren beträgt die durchschnittliche Laufzeitkomplexität schlussendlich $\mathcal{O}(nP \cdot nA)$. Die Speicherkomplexität lässt sich mithilfe der Formel $nP + nA$ darstellen. Das liegt daran, dass man `nP` Listen für `groups` braucht und gleichzeitig `nA` Werte auf die Listen verteilt werden. Durch die Verteilung wachsen die Werte nicht proportional zueinander, weshalb man es als eine Summe schreiben kann. Schlussendlich beträgt die durchschnittliche Speicherkomplexität $\mathcal{O}(nP + nA)$, da aber standardmäßig $nA \gg nP$ ist, kann man es zu $\mathcal{O}(nA)$ zusammenfassen.

In einem Best-Case Szenario (wenn $nA == 1$ ist) beträgt die Laufzeitkomplexität und Speicherkomplexität $\mathcal{O}(1)$.

In `totalSize` beträgt die Laufzeitkomplexität in einem Worst-Case Szenario $\mathcal{O}(nA \cdot nP \cdot nK)$, wobei `nK` für die Länge der Kodierung steht. Das liegt daran, dass zwei Schleifen verwendet werden, die für

alle Perlendurchschnitte und alle Symbole durchgehen. Für jeden Durchgang wird zusätzlich noch die Funktion `count` bei der Kodierung ausgelöst, weshalb man den Faktor nK zusätzlich noch beachten sollte. Da aber meistens $nP \ll nA$ und $nK \ll nA$ ist, dürfte man es auch durchschnittlich $\mathcal{O}(nA)$ darstellen. Die Speicherkomplexität hingegen beträgt $\mathcal{O}(1)$, da nur `total` als Variable gespeichert wird und alle Operationen iterabel sind, weshalb man keine Zwischenspeicherung der Datensätze benötigt.

Im `main` werden mehrere Funktionen ausgelöst. Da `main` die Funktion `getPrefix` auslöst, übernimmt es dadurch die Laufzeitkomplexität der Funktion. Das liegt daran, dass `main` selber $\mathcal{O}(\text{content} + nA + nA \log nA + nP)$ groß sein kann, wobei `content` für die Länge der Beispieldatei steht. Die Aussage $nA \log nA$ entsteht durch das Sortieren aller Häufigkeitswerte. Da meistens $nA \log nA \ll nA \cdot nP$ gilt, beträgt die durchschnittliche Laufzeitkomplexität $\mathcal{O}(nA \cdot nP)$. Im Bereich der Speicherkomplexität beträgt diese Funktion in einem Worst-Case Szenario $\mathcal{O}(\text{content} + nA + nP)$. Das liegt daran, dass der Inhalt des Textes gespeichert werden muss und gleichzeitig auch noch die Anzahl der Perlen sowie die Häufigkeit der einzelnen Symbole. Da aber meistens $nP \ll \text{content}$ und $nP \ll nA$ gilt, lässt es sich auf $\mathcal{O}(\text{content} + nA)$ vereinfachen, was auch die durchschnittliche Speicherkomplexität beträgt.

4.1 Analyse der Problemstellung

Es ist schwierig zu sagen, wie man dieses Problem einstufen soll. Jedoch bin ich auf jeden Fall überzeugt, dass diese Problemstellung kein Teil von P ist. Wenn die einzelnen Perlen eine eigene Gewichtung erhalten, ergibt sich ein massives Problem: Wenn man sich diese Problemstellung als ein Baumdiagramm vorstellt, gibt es absolut keine Möglichkeit, eine eindeutige Aussage für mathematische oder logische Entscheidungen zu treffen, wenn sich die Knoten räumlich voneinander getrennt sind und mithilfe einer bzw. mehrerer Parent-Knoten miteinander verbunden sind. Dieses Problem existiert beim Lösen für alle mir bekannten (bzw. die ich mir in den letzten 4 Monaten angeschaut habe) Lösungsansätze. Ich persönlich habe schon bereits versucht, mithilfe von AVL-Bäumen, gewichteten binären Suchbäumen, Rot-Schwarz-Bäumen, Huffman-Codierung sowie Shannon-Fano-Codierung einen Ansatz zu entwickeln, mit dem man die Präfixcodes für die kürzeste Kettenlänge auf polynomieller Zeit berechnen kann, jedoch ist es unmöglich, ohne durch willkürliches Raten eine Codierung zu finden. Bei den AVL-Bäumen gibt es keine Möglichkeit, eine Aussage über die Optimierungsmöglichkeit zu erstellen, wenn man versucht Häufigkeit sowie Perlendurchmesser einzubauen, da die Aussagen, die ich berechnet habe, mir komplett willkürlich erschienen. Bei der Huffman-Codierung sowie der Shannon-Fano Codierung gibt es das Problem, dass es auf polynomieller Zeit nicht möglich ist, die Perlendurchmesser einzubauen und dabei die optimale Lösung zu bekommen.

Wenn man darüber entscheiden müsste, ob ein Symbol, welches sich in einer Gruppe bzw. in mehreren verschachtelten Gruppen befindet, für einen bestimmten Text wirklich die optimale Stellung besitzt, kann man keine Aussage mit polynomiellem Ansatz treffen. Durch die Verschachtelung der Knoten kann kein mathematischer bzw. logischer Ansatz in polynomieller Zeit gebracht werden, der diese zwei Aussagen gleichzeitig beantworten kann: Ist diese Baumstruktur optimal? Und ist diese Knotenverteilung optimal? Der einzige Weg, den ich hier sehe, ist das Brute-Forcing.:

Für eine einfache Gleichung lässt sich folgende Gleichung erstellen:

$$\text{Gesamthäufigkeit der Knoten} \cdot \text{Das Gewicht des Weges} = \text{Teilgewichtskosten} \quad (3)$$

Dabei ähnelt diese Gleichung der Problemstellung einer kryptografischen Funktion: Wenn man einen bestimmten Wert bzw. in diesem Fall das niedrigste Teilgewicht nicht kennt, ist der einzige Weg, alle Möglichkeiten zu probieren, bis man den Wert gefunden hat. Jedoch kommt noch hinzu: Besitzt dieser Weg wirklich die optimale Struktur? Lässt sich für die Häufigkeiten ein besser optimierter Weg bilden? Die Kombination dieser zwei Problemstellungen macht es schlussendlich meiner Meinung nach unmöglich, einen optimalen Weg auf polynomieller Zeit zu bilden.

Wenn man eine Kodierung mit einer optimalen Symbolverteilung hat, kann es sein, dass die Struktur nicht optimal ist. Würde man Änderungen an der Struktur vornehmen, so kann es sein, dass die Symbolverteilung wieder nicht optimal wird. Ändert man nun die Symbolverteilung, so kann es wieder sein, dass diese Struktur nicht optimal wird.

Um eine Antwort auf beide Aussagen zu geben, muss man mit einer Brute-Force raten, nämlich auf alle möglichen Strukturen als auch Verteilungsmöglichkeiten. Die Möglichkeiten, die sich daraus ergeben, nehmen mit mehr Symbolen als auch Perlenarten exponentiell zu. Aus diesem Grund lassen sich Lösungen ebenfalls nicht in polynomieller Zeit beantworten, da ein Ansatz nötig ist, der mit mehr Eingaben exponentiell länger dauert. Diese Problemstellung ist deshalb kein Teil von NP, weshalb ich es unter NP-Hard einordne.

5 Beispiele

Für die Beispiele habe ich beide Algorithmen verwendet. Aus Platzgründen sind alle vollständigen Lösungen im Ordner zu finden. Alle Dateien, die mit dem Namen „ERGENIS_default“ beginnen, wurden mithilfe der ersten Lösung bzw. Originallösung gelöst. Dateien, die mit „ERGENIS_huffman“ beginnen, wurden mithilfe der modifizierten Huffman-Codierung gelöst. Dabei wird deutlich, dass das Huffman Verfahren vor allem bei Aufgaben, die einen gleich großen Durchmesser von Perlen besitzen (schmuck0, schmuck00 und schmuck01), die bestmögliche Lösung ausgibt. Durch die Verwendung von UTF-8 als Kodierung können alle Dateien eingelesen und gespeichert werden.

Für die Berechnungen mithilfe der modifizierten Huffman-Codierung wurden die Ergebnisse für alle Aufgaben in unter 0.1 Sekunden berechnet. Bei der Originallösung wurden alle Aufgaben in unter 16ms berechnet.

Beispielsdateiname	Originallösung Ausgaben	Huffman Codierung Ausgaben
schmuck0.txt	114 -> 11.4 cm	113 -> 11.3cm
schmuck00.txt	379 -> 37.9 cm	372 -> 37.2cm
schmuck1.txt	197 -> 19.7 cm	
schmuck01.txt	1262 -> 126.2 cm	1150 -> 115.0cm
schmuck2.txt	158 -> 15.8 cm	
schmuck3.txt	279 -> 27.9 cm	
schmuck4.txt	138 -> 13.8 cm	
schmuck5.txt	3178 -> 317.8 cm	
schmuck6.txt	238 -> 23.8 cm	
schmuck7.txt	143303 -> 14330.3 cm	
schmuck8.txt	3361 -> 336.1 cm	
schmuck9.txt	36984 -> 3698.4 cm	

6 Quellcode

```
def getPrefix(codes, amounts, prefix, dPerle, weight = 0):
    if len(amounts) == 1:
        codes[prefix] = weight
        return

    groups = [[] for i in dPerle] # Eine Liste/Gruppe pro Perlenart
    sums = [0] * len(dPerle)

    # Jede Häufigkeit in eine Gruppe zuordnen
    for amount in amounts:
        index = sums.index(min(sums)) # Kleinste Werte priorisieren
        groups[index].append(amount)
        sums[index] += amount * dPerle[index]

    # Für jede Gruppe die Häufigkeiten in weitere Untergruppen zuordnen.
    for i in range(len(dPerle)):
        if groups[i]:
```

```

        getPrefix(codes, groups[i], f"{prefix}{i}", dPerle, weight + dPerle[i])

def totalSize(prefixcodes, charmap, dPerle):
    total = 0 # Gesamtgröße
    # Für alle verschiedenen Zeichen durchführen
    for key in prefixcodes.keys():
        # Für jede Perlenart
        for i in range(len(dPerle)):
            # Anzahl der verwendeten Perlen * Gesamtzahl Häufigkeit des Zeichens * Perlengröße
            total += prefixcodes[key].count(f"{i}") * charmap[key] * dPerle[i]
    return total, f"{total/10} cm"

def main(filename):
    # Datei ablesen
    with open(f"Beispiele\\{filename}", encoding="utf-8") as f:
        nPerle = int(f.readline()) # Anzahl verschiedene Perlen
        dPerle = [int(i) for i in f.readline().split(" ")] # Perlendurchmesser
        sentence = f.readline().strip("\n")

    # Anzahl Buchstaben zählen
    charmap = {} # Buchstabe: Häufigkeit
    charset = set() # Hashmap statt Kontrolle über Listen, um die Laufzeit zu verringern
    for letter in sentence:
        if letter in charset:
            charmap[letter] += 1
        else:
            charset.add(letter)
            charmap[letter] = 1

    # Symbole und Häufigkeiten in zwei verschiedenen Listen trennen, wobei es die Reihenfolge
    der Häufigkeiten weiterhin behält.
    # Es wird absteigend abhängig von der Häufigkeit sortiert.
    # Es gilt x[a] >= x[b] für int a < int b
    charlist = list(i for i in charmap.items())
    charlist.sort(key=lambda i: i[1], reverse=True)
    amounts = [i[1] for i in charlist] # Alle Häufigkeiten sortiert (absteigend)
    symbols = [i[0] for i in charlist] # Alle Zeichen sortiert (absteigend)

    codes = {} # Kodierung : Gewicht
    getPrefix(codes, amounts, "", dPerle)

    # Alle Kodierungen nach deren Gewicht sortieren (aufsteigend)
    codesorted = dict(sorted(codes.items(), key=lambda i: i[1]))

    # Für alle Symbole (nach der Häufigkeit absteigend sortiert) eine Kodierung (nach dem
    Gewicht aufsteigend sortiert) zuweisen
    NewCode = {} # Symbol : Kodierung
    for i, key in enumerate(codesorted.keys()):
        NewCode[symbols[i]] = key

    # Prefixcode ausdrucken
    amount, length = totalSize(NewCode, charmap, dPerle)
    with open(f"Ergebnis\\ERGEBNIS_default_{filename}", "w", encoding="UTF-8") as f:
        f.write(f"{filename}\nTotalsize: {amount}\nConverted in length: {length}\n")
        for key in charmap.keys():
            f.write(f"{key} {charmap[key]}: {NewCode[key]}\n")

    #print(f"{filename}: {amount} -> {length}")
    print(f"{amount} -> {length}")

```

6.1 Huffman-Codierung

```

class Knoten:
    def __init__(self, letter, amount, childrens=None):
        # Knoten erstellen
        # letter: Buchstabe oder Nichts
        # amount: Häufigkeit der Buchstaben
        # childrens: Untere Knoten
        self.letter = letter
        self.amount = amount
        if childrens is None:
            self.childrens = []
        else:
            self.childrens = childrens

    def __lt__(self, other):
        return self.amount < other.amount

    def totalSize(self, weight):
        # Gesamtgröße zurückgeben
        if not self.letter is None:
            return self.amount * weight
        sum = 0
        for i in range(len(self.childrens)):
            sum += self.childrens[i].totalSize(weight + 1)
        return sum

    def strOut(self, code=""):
        # Alle Huffman-Codes zurückgeben
        text = ""
        if not self.letter is None:
            text = f"{self.letter} ({self.amount}): {code}\n"
        for i in range(len(self.childrens)):
            text += self.childrens[i].strOut(code + str(i))
        return text

    def printOut(self):
        # Alle Huffman-Codes ausdrucken
        print(self.strOut())

def main(filename):
    # Datei Lesen
    with open(f"Beispiele\\{filename}", encoding="utf-8") as f:
        nPerle = int(f.readline())
        dPerle = [int(i) for i in f.readline().split(" ")]
        sentence = f.readline().strip("\n")

    # Anzahl Buchstaben zählen
    charmap = {}
    charset = set()
    for letter in sentence:
        if letter in charset:
            charmap[letter] += 1
        else:
            charmap[letter] = 1
            charset.add(letter)

    # voraussichtliche Überschuss berechnen
    Joker = (len(charmap) - 2) % (nPerle - 1) + 2
    #if Joker == nPerle:
    #    Joker = 0

    # Knoten erstellen

```



```
Knoten = [Knote(i, charmap[i]) for i in charmap.keys()]
Knoten.sort(reverse=True)

while len(Knoten) > 1:
    # Huffman Codierung anwenden
    childrens = [Knoten.pop() for i in range(Joker if Joker > 0 else nPerle)]
    Joker = 0
    amount = 0
    for child in childrens:
        amount += child.amount
    KnotenNeu = Knote(letter=None, amount=amount, childrens=childrens)
    Knoten.append(KnotenNeu)
    Knoten.sort(reverse=True)

# Speichern
totalSize = Knoten[0].totalSize(0)
with open(f"Ergebnis\\ERGEBNIS_huffman_limited_"+filename, "w", encoding="utf-8") as f:
    f.write(f"{filename}\nTotalsize: {totalSize}\nConverted in length: {totalSize/10}
cm\n")
    f.write(Knoten[0].strOut())
print(f"{filename}: {totalSize} -> {totalSize/10}cm")
```